

Appendix A:

ODD Model Description of the SpaDES *anthropogenicDisturbance* Set of Modules

Model: SpaDES Anthropogenic Disturbance Simulation (*anthropogenicDisturbance*)

Protocol: ODD (Grimm et al., 2006; 2010; 2020)

Date: 2026-01-06

Prepared by: Ronald Kilian

Table of Contents

ODD Model Description of the SpaDES <i>anthroDisturbance</i> Set of Modules.....	1
1. Purpose and Patterns.....	2
2. Entities, State Variables, and Scales.....	3
3. Process Overview and Scheduling.....	5
4. Design Concepts.....	8
5. Initialization.....	12
6. Input Data.....	14
7. Submodels.....	20
7.1 Data Harmonization (Disturbance Data Prep).....	20
7.2 Potential Resource Layer Combination (Idiosyncratic Data Prep).....	22
7.3 Disturbance Parameter Calculations.....	23
7.4 Disturbance Generation Dynamics.....	24
7.5 State update and outputs.....	26

1. Purpose and Patterns

Purpose: The model simulates the spatiotemporal expansion of anthropogenic disturbances (e.g. settlements, roads, forestry cutblocks, mines, oil/gas wells) in a northern landscape, using a modular SpaDES framework. It is intended primarily for the Northwest Territories (Canada) boreal region, in support of ecological studies (e.g. boreal caribou habitat scenarios), but the structure is generalizable to other regions given appropriate inputs. The simulation provides a scenario tool to project future human footprint under different assumptions, ensuring that the model's behavior reproduces real-world patterns of disturbance growth and arrangement as closely as possible.

Target Patterns: Following Grimm et al. (2020), we identify key spatial and temporal patterns from data to evaluate model realism. First, the model is calibrated to match observed rates of new disturbance: for example, known increases in total disturbed area between 2010 and 2015 (from Environment Canada's national anthropogenic footprint maps) inform the annual disturbance percentages for each sector. A broadly used guideline - no more than ~0.2% of the area newly disturbed per year for caribou ranges - is also considered as a reference pattern. Second, the model reproduces spatial clustering patterns of linear disturbances: real seismic lines often form grid-like clusters, so the model characterizes line networks in the input data and uses those patterns (e.g. typical line length, spacing, and angular alignments) to generate new lines with similar geometry. In general, each disturbance type's simulated behavior is grounded in empirical patterns (e.g. the tendency of new mines to appear in the highest mineral potential areas, or new roads to extend existing road networks) which serve as design criteria for the model's rules.

2. Entities, State Variables, and Scales

Entities and State Variables: The model represents a spatially explicit environment and various disturbance features within it. Key entity types include:

- **Landscape grid (environment):** The study area is defined by a polygon and a matching raster grid (e.g. 250 m resolution) that all spatial data align to. This provides the spatial extent (e.g. the NT1 caribou range plus adjacent region) and resolution for the simulation. State variables for the environment include the total area (used to compute disturbance percentages) and spatial masks like a Digital Elevation Model or water bodies (optional inputs to constrain development). Time is modeled in discrete year steps; by default the simulation starts at year 2011 (the last year with “current” data) and runs to a chosen end year (e.g. 2050), with events typically scheduled at multi-year intervals (see scheduling below).
- **Disturbance classes (by sector/type):** Each type of anthropogenic disturbance is an entity characterized by its spatial layer of current disturbances. Disturbance classes correspond to industries or feature types (e.g. Settlements, Forestry cutblocks, Mines, Wind turbines, Roads, Pipelines, Seismic lines, etc.), matching the categories in the input metadata. For each class, the model tracks a spatial object (polygon or line features, or raster) representing all currently disturbed areas of that type. These layers are organized in a nested list structure by sector and disturbance type (after initial data prep). Important state variables for each disturbance class include its total disturbed area and attributes like feature geometry. Some classes have associated “potential” layers (see below) that indicate where new disturbances of that type *could* occur.
- **Potential development layers:** For disturbance types that involve future expansion into currently undeveloped areas, the model uses separate “potential” spatial layers stored alongside current disturbance layers within `disturbanceList`. In the default Northwest Territories workflow, these potential layers are typically polygon layers (`SpatVector`) carrying a suitability/priority value used for siting new features (standardized to a field named `Potential` inside the generator). Higher Potential values represent locations that are selected earlier/more often for new development. Some potential inputs may originate as rasters (e.g., prospectivity surfaces), but are represented as spatial polygons after preprocessing so the generator can sample locations within suitable areas.

- **Disturbance parameters:** A global table `disturbanceParameters` defines the processes simulated for each disturbance class. Each row specifies (at minimum) `dataName` (sector), `dataClass` (the associated “potential” layer name), `disturbanceType` (one of Enlarging, Generating, Connecting), `disturbanceOrigin` (the class being modified/created/used as the origin for connections), `disturbanceEnd` (the destination network layer for Connecting types), and `disturbanceInterval` (years between events). For Generating and Enlarging types, `disturbanceRate` is an annual percentage of the total study area (in percent units, e.g., 0.2 = 0.2%/year). For Generating types, `disturbanceSize` is stored as an R expression string evaluated during simulation to sample an area in m² for each new feature (e.g., a truncated normal draw). For Connecting types, `disturbanceRate` and `disturbanceSize` are not used (typically NA). The `resolutionVector` field is used when buffering points/lines to polygons for merging and masking.

Scales: Spatially, the model operates on the scale of the entire study region divided into grid cells of 250 m (or another resolution) - fine enough to represent individual disturbance features (e.g. a well pad or road segment covers multiple 250 m cells). All spatial operations (overlap, buffering, etc.) respect a common coordinate reference system and resolution given by the `rasterToMatch` input. Temporally, time steps are measured in years. The model can advance in one-year increments, but by default it schedules disturbance events at a multi-year interval (e.g. every 10 years) for efficiency. For instance, with a 10-year step, the simulation might progress from 2011 → 2021 → 2031 ... up to the end year, applying disturbance growth in each interval. The choice of interval can be adjusted (parameter `runInterval`) to match the frequency of disturbance pulses or data availability. There is no adaptive spatial scaling or nesting of grids beyond the single resolution; however, different disturbance processes inherently act at different scales (e.g. local buffering of a town vs. broad distribution of new well sites).

3. Process Overview and Scheduling

Overview of Processes: The simulation consists of three interlinked SpaDES modules - Data Preparation, Potential Resources (idiosyncratic data prep), and Disturbance Generator - each with its own internal events. The modules operate in sequence at the start, then the generator module runs an iterative event loop over time (the core yearly/decadal simulation). Major processes include: loading and harmonizing various input disturbance datasets into a common format; combining or adjusting specific “potential” layers (e.g. merging multiple mining potential datasets into one); calculating any missing disturbance parameters (such as the rate or size of new disturbances based on historical data patterns); generating new disturbances at each time step (including enlarging existing ones, adding new ones in suitable locations, and connecting them via linear infrastructure); and updating the state for the next cycle. These processes and their scheduling are summarized in Figure 1 and described below.

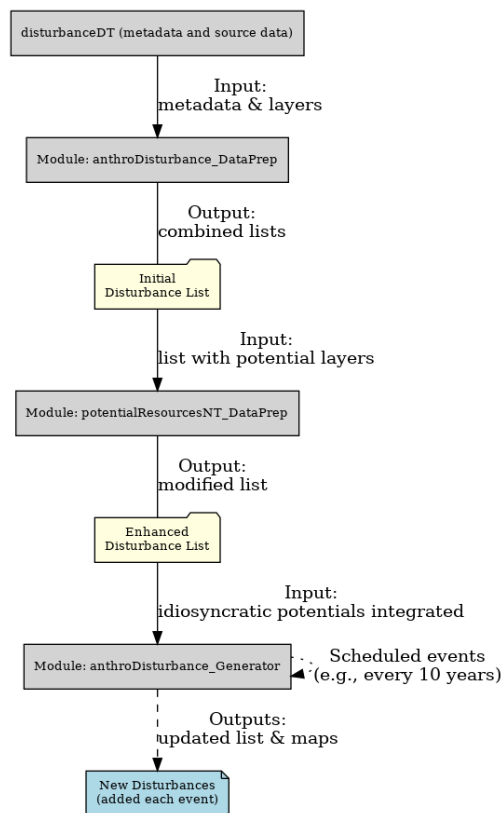


Figure 1: Simplified structure of the SpaDES simulation event flow, showing how the three modules interact and pass data. The anthroDisturbance_DataPrep module runs first, reading raw disturbance datasets via the disturbanceDT metadata table and outputting a nested list of spatial disturbance layers (disturbances and a harmonized disturbanceList). Next, the potentialResourcesNT_DataPrep module modifies this list, combining

multiple “potential” layers into refined potential surfaces for oil/gas and mining development. Finally, the `anthroDisturbance_Generator` module is initialized with the prepared disturbance list and parameters, and enters a recurring loop of events (scheduled at a specified interval, e.g. every 10 years) to simulate new disturbances. At each interval, it produces updated disturbance layers and summary outputs (maps), which feed back into the state for the next cycle.

SpaDES Scheduling: The model uses SpaDES’s discrete-event simulation scheduler to manage the timing and order of events. When the simulation is initialized (`simInit`), each module inserts its init event into the event queue at time = `start(sim)` (the start year). Module initialization events have default priority 1, and subsequent events can be assigned higher priorities to enforce ordering when they share the same time. In this model, the `DataPrep` module init runs first at time T_0 (start year), then schedules its main data loading event immediately (at the same time T_0) with priority 2. The `PotentialResources` module init runs next and schedules its series of combination events at T_0 with priority 3. Finally, the `Generator` module init runs and schedules the first set of generation events at T_0 with priority 4. This ensures the data are prepared in the correct order before generation begins.

After the initial time step, the `anthroDisturbance_Generator` module is responsible for all recurring events. In its init, it schedules four key processes - `calculatingSize`, `calculatingRate`, `generatingDisturbances`, and `updatingDisturbanceList` - to occur at the current time (`start`) with the specified priority. These will execute in order (`size` → `rate` → `generate` → `update`) during the first time step. At the end of each cycle, the generator schedules the next occurrence of the `generatingDisturbances` and `updatingDisturbanceList` events at time = current time + `runInterval`. Thus, if `runInterval` = 10 years, after processing year 2011 the module schedules the next generation at 2021, then 2031, and so on. This forms a loop where at every scheduled time, new disturbances are added and the disturbance list updated. The loop continues until the simulation’s end time is reached, at which point no further events are scheduled. The event sequence in each time step is: parameter calculation events (only needed at the first step), generation of disturbances, and updating the state and re-scheduling. All events are executed synchronously at the year they are scheduled, making the model a time-stepped simulation with discrete-event implementation.

Throughout the simulation, certain events may be conditional or skipped depending on the state. For example, the `calculatingSize` event will do nothing (skip) if all disturbance size parameters are already provided by the user. Similarly, `calculatingRate` only runs if there are disturbance types missing rate values. The scheduling logic thus allows the model to default to user-supplied parameters or to compute them from

data if needed. There are also protective checks; for instance, the PotentialResources module will terminate with an error if the simulation's start year is set before 2011 (since required baseline data would be unavailable).

4. Design Concepts

Basic Principles: This model follows a pattern-oriented modeling approach. Rather than modeling human decision-making in detail, it imposes rules and parameters chosen to reproduce observed patterns of disturbance on the landscape. For instance, the principle that linear disturbances form grid patterns is captured by statistical rules for line clustering, and the principle that development tends to start in the most favorable locations is captured by using ranked potential surfaces and an “exhaustive” site selection option (picking highest-quality sites first). The model’s design is grounded in landscape ecology and conservation planning: disturbances are treated as spatial features that grow or appear according to empirical rates and constraints (e.g. resource potential, proximity to infrastructure, avoidance of water bodies), ensuring that model outcomes are realistic for habitat disturbance studies. There are no explicit agent-based decision heuristics; instead, the model operates as a stochastic landscape simulator informed by GIS data and user-specified scenarios. Key theoretical assumptions include: future disturbance expansion will be similar in character to past trends (stationarity in patterns of growth), and spatial drivers like resource potential and accessibility are sufficient to predict where new disturbances will occur.

Emergence: Higher-level patterns in this model emerge from the cumulative effect of many local disturbance additions. For example, the total disturbed area over time is an emergent result of the yearly additions (though guided by target rates). Similarly, the spatial configuration of disturbances - such as the connectivity of road networks or the dispersion of new cutblocks - is not predetermined but results from the model’s rules (shortest-path connections, random site selection, etc.) interacting with the existing landscape. The hope is that large-scale patterns (e.g. proportion of landscape disturbed, or fragmentation patterns) will align with observed or expected patterns as criteria for model validation. Some outcomes are imposed by design (e.g. a specified disturbance rate will lead to a corresponding area disturbed), but how that area is distributed spatially and among disturbance types is an emergent property influenced by the competition for space and random draws. Notably, connectivity patterns emerge: as new linear features are added via shortest-path algorithms, complex networks of access roads or pipelines form over time without being explicitly hard-coded - they arise from connecting new points to the nearest existing network. The model does not incorporate adaptive agent behaviors, so social or economic patterns are not emergent here; only biophysical disturbance patterns emerge.

Adaptation and Objectives: *Not applicable.* There are no agents with adaptive behavior or explicit objectives in this model. The “agents” can be thought of as the disturbance processes themselves, but they do not learn or change strategies - they follow fixed rules. Stakeholder decisions (e.g. a company’s adaptive strategy) are not represented; instead, their effect is approximated by static rules (e.g. always develop highest potential first or spread out development probabilistically). Thus, adaptation, learning, and goal-driven behaviors are not modeled.

Sensing and Interaction: There are no cognitive agents, but the model’s processes do have a form of interaction with the landscape state. For instance, the disturbance generation process “senses” the current configuration of disturbances by checking where existing disturbed cells are, to avoid placing new disturbances on them (i.e. new features are only sited on currently undisturbed cells). In a sense, the algorithmic processes respond to the evolving state: e.g. the connecting algorithm finds nearest existing roads, implicitly sensing the road network; the forestry module can check for recent burns (fire disturbance) to avoid cutting there. Interactions in the model occur implicitly between disturbance types: for example, the creation of a new mine (point disturbance) triggers the need for a road (linear disturbance) to connect it to infrastructure. This is handled by the connecting submodel which “interacts” new generating-type features with existing linear features to integrate them into one network. However, there are no direct agent-agent interactions - the interactions are process-based (one module’s output feeds another’s input). Additionally, when multiple disturbance types overlap in space (e.g. a cutblock overlapping a seismic line), the model can resolve overlaps by masking one type out of another if configured (to prevent double-counting area).

Stochasticity: The model incorporates randomness in several processes, reflecting uncertainty and variability in how disturbances expand. Many parameters are treated probabilistically. For example, if the typical size of a new disturbance is unspecified, the model will derive a probability distribution (e.g. a truncated normal) from data and then draw random sizes for each new feature. The placement of new disturbances can be stochastic: under the “distributing” site selection mode, each potential development pixel has a probability of being chosen proportional to its potential value. This means repeated runs of the model yield different specific maps of disturbances, even if the overall rates are the same. Random seeds can be set per event (the modules accept .seed parameters) to ensure reproducibility if desired. Random clustering of seismic lines as an example: the model may randomly choose one of several empirically-derived line cluster “patterns” for each new development area. Overall, random draws

are used whenever fine-scale outcomes are not predetermined by the inputs, ensuring the model is not over-fitted and can produce a range of plausible disturbance landscapes.

Collectives: There are no explicit collective behaviors (no groups of agents). However, the model does consider collections of features in some calculations - for instance, clusters of line features are identified and characterized as a group during initialization to inform how new line grids will be generated. One could view each disturbance type as a collective of features that jointly contribute to area and pattern outcomes. The model treats entire classes (e.g. all roads) in aggregate when applying rates or buffering (e.g. enlarging all settlements by a buffer) - this aggregate approach is a simplification, not a true collective behavior emerging from interactions. Thus, aside from grouping features by type, the concept of collective decision-making does not apply.

Observation: The model produces a variety of outputs for the purpose of monitoring its behavior and enabling validation. At each time step, the current state of disturbances can be recorded. The simulation maintains a list `sim$currentDisturbanceLayer` which stores a raster map of all disturbances for each output year. These maps can be used to visualize the cumulative footprint over time or to compute statistics (e.g. percent area disturbed). If the user enables output saving, the model will write these disturbance rasters to files at each step (e.g. naming them by year and disturbance type). The model also logs console messages for important events - for example, after each disturbance generation it may report the achieved vs. expected area disturbed percentage. In addition, the final `disturbanceList` (which contains all spatial features by class) is an output that can be saved or examined to see exactly what features were added. No specific formal data assimilation or online calibration occurs, but the outputs are designed to be compared with the input patterns: e.g. the user can compare the model's disturbed area trajectory to the historical 2010-2015 pattern, or check if the final total disturbance meets a policy target. In summary, the model's observables include spatial outputs (maps of disturbance per year), summaries (areas by type), and any diagnostic tables (e.g. a proportion of area disturbed by buffered vs. unbuffered measure, if `checkDisturbanceProportions` was enabled to output such info). These allow the modeler to verify that the simulated patterns align with real patterns and the purpose for which the model was built.

5. Initialization

Initial State: The simulation begins at a specified start year (typically 2011 for the Northwest Territories case, as 2011 is the baseline year of “current” disturbance data). At time T_0 (start), the two data preparation modules create the initial conditions. The `anthroDisturbance_DataPrep` module loads all provided disturbance datasets (historical disturbance footprint maps, infrastructure layers, etc.) and produces a unified data structure. Specifically, it reads the `disturbanceDT` metadata table (see Input Data) and downloads or opens each listed dataset (e.g. shapefiles of roads, settlements, etc.), then projects and crops them to the given study area and raster extent. All disturbances present at the start year are thus represented as spatial features in memory. These features are organized into a nested list `sim$disturbances` (outer list by sector `dataName`, inner list by `dataClass`). Next, a function `harmonizeList` (called within `DataPrep`’s main event) processes this list to combine or refine classes: if multiple datasets represent the same disturbance class, they are merged into one layer, and any classes marked “potential” are kept separate (not merged with current disturbances). The result is a cleaned `sim$disturbanceList` where each disturbance class (except special potential cases) has a single composite layer ready for simulation. For example, if the metadata listed two road datasets, they would now be combined into one “Roads” layer in `disturbanceList`. By the end of `DataPrep`’s `init` and `loadAndHarmonizeDisturbanceDT` events, the model has in memory: the study area polygon, a reference raster, and a `disturbanceList` of all current disturbance features categorized by type.

The `DataPrep` module supports on-disk reuse of already-processed layers: during data preparation it writes study-area-cropped layers to disk (keyed by the study area) and reloads those files on subsequent runs when they exist. In the current implementation, running the `DataPrep` module still executes its load-and-harmonize event; skipping `DataPrep` is accomplished by supplying a prepared `disturbanceList` directly and omitting the `DataPrep` module from the simulation.

Before any disturbances are grown, the `Generator` module `init` performs some final checks and setups. It verifies that certain parameters are valid (e.g. ensuring any `siteSelectionAsDistributing` flag, which toggles the disturbance placement mode, matches one of the disturbance origin types). It aligns the `studyArea` and `rasterToMatch` projections (though this should already be handled in `DataPrep`). It also ensures the `disturbanceDT` metadata is available as a `data.table` (used for reference in some calculations). Critically, the generator initializes `sim$currentDisturbanceLayer <-`

list(), which stores the disturbance layers returned by each generation step. In the default vector workflow, each entry (e.g., `sim$currentDisturbanceLayer[["Year2021"]]`) is a nested list of spatial layers (primarily `SpatVectors`), organized by sector and class. These layers are used as an exclusion mask for subsequent site selection (e.g., to erase already-disturbed areas from potential polygons for most disturbance types). The model can additionally write buffered raster summaries (e.g., 500 m buffered disturbance masks) as separate outputs when requested. By default `disturbFirstYear = FALSE`, so no new disturbances are added at `start(sim)`; generation begins at the next scheduled event time. If `disturbFirstYear = TRUE`, the generator will also create new disturbances at `start(sim)`. g. year 2021, if start is 2011 and interval 10). To summarize, after initialization the model state reflects the real-world disturbance landscape at the start year, and all data structures are in place for the simulation to proceed. The total undisturbed area, as well as each class's distribution, is known from the input data. The initial disturbance patterns are effectively a starting point that the model will build upon.

6. Input Data

Because this model integrates numerous data layers, careful specification of input data is required. All input datasets are documented in a metadata table called `disturbanceDT`, which is the primary input (provided as an R `data.table` or CSV). Each row of `disturbanceDT` describes one data source and how it should be processed. Key columns include:

- **dataName:** The general sector or category of the disturbance (e.g. "Energy", "Forestry", "Mining", "Settlements", "Transportation"). This is used as the name of the top-level list in outputs. Multiple entries can share the same `dataName` if they are different datasets for the same sector.
- **dataClass:** A specific disturbance class name within that sector (e.g. "roads", "cutblocks", "oilWells", "windTurbines", "seismicLines", "pipelines", etc., including "potential" classes). This defines the sub-category/list within the sector. The `dataClass` is what the model ultimately uses to distinguish types of disturbances. If multiple data sources represent the same class, they will be merged. For "potential" classes, this field typically contains the word "potential" (e.g. "potentialMining") to flag special handling.
- **URL:** A link or file path to the data source. The module will download or read this file. URLs can point to shapefiles (as zip archives or individual files), geodatabases, raster files (e.g. .tif), or other supported formats. For example, there might be a URL to a government open data portal for linear disturbance shapefiles, or to a Google Drive file. The model has default URLs for the NW Territories data built-in (for instance, a default `disturbanceDT.csv` is provided via GitHub raw link, and default spatial data like a study area shapefile and raster via Google Drive).
- **classToSearch and fieldToSearch:** Optional fields used if the source dataset contains multiple feature classes and a subset needs to be extracted. For example, if a shapefile contains many disturbance types in one, `fieldToSearch` might be "DisturbType" and `classToSearch` could be "Road" to select only road features from it. If provided, the DataPrep module will filter the spatial data to only those entries matching the given class in the specified attribute field.

- **fileName:** If the data is inside an archive (zip) and not named obviously, this specifies the exact filename to use within the zip. This is needed for certain downloads where multiple files are packaged together.
- **dataType:** The format of the data (e.g. 'shapefile', 'raster', 'mif'). This guides how the data will be read (e.g. using `terra::vect` for vector or `terra::rast` for rasters). The module supports converting rasters to polygons if needed (e.g. a binary disturbance raster could be converted to vector) and reading MapInfo .mif files as shapefiles.

This disturbanceDT is crucial - it essentially tells the model what data to use and how. In practice, for the thesis case, this table would list all anthropogenic disturbance datasets for NW Territories: e.g., an entry for the 2015 human footprint raster (from Environment and Climate Change Canada), entries for each type of industrial disturbance (mines, wells, cutblocks, etc.) with their source shapefiles, and entries for each “potential” layer (like geology-based potential maps or planned development layers). An example excerpt: one row might have `dataName = "Mining"`, `dataClass = "potentialMining"`, URL pointing to a mineral potential map, and another row with `dataName = "Mining"`, `dataClass = "mines"`, URL pointing to a shapefile of existing mine sites. The DataPrep module would load both, merge existing mines into one layer and keep potentialMining separate.

Other input objects required include:

- **Study area (studyArea):** A polygon delineating the modeling area. All outputs will be clipped to this area. If not supplied, the modules will default to a predefined Northwest Territories + boreal region shape. The study area is used to crop input data, to calculate total area for percentage computations and to set the CRS.
- **Raster-to-match (rasterToMatch):** A raster layer defining the target resolution, extent, and projection for the spatial data. All disturbance layers are transformed to match this raster’s grid. If not given, a default 30 m raster for the region is loaded.
- **Disturbance parameters table (disturbanceParameters):** If the user has a custom CSV or data table specifying disturbance types and their parameters (rate, size, etc.), it can be provided. Otherwise, the model will load a default parameter set included with the module (for example, an internal data file `paramsGeneral.txt` for

NT). This table, as described earlier, guides the simulation rules for each type. Users might tweak this input to run different scenarios (e.g. increasing the rate for oil/gas development to simulate a boom).

- **Potential-combination instructions (whatToCombine):** A small data table input (or parameter) used by the PotentialResourcesNT module, specifying how to combine layers. By default, the module has a built-in table for NW Territories where, for instance, it knows to use CLAIM_STAT and PERMIT_STA fields for mining, and a certain raster for oil potential. If the user were adapting the model to a new region, they might provide a modified whatToCombine to reflect available data (or skip this module if not needed).
- **Fire layer (rstCurrentBurn) (optional):** A raster of recent burns (forest fire disturbance) for the study area. This is only needed if one wants the forestry disturbance process to avoid areas recently burned. If provided (or generated by a fire simulation module), the generator will use it to mask out those areas when placing new forestry cutblocks. If not supplied, the model will assume no fire disturbances (or will attempt to find one in an input folder, as indicated by a warning).
- **Features-to-avoid mask (featuresToAvoid) (optional):** A spatial layer (often a raster) indicating areas where new linear features should not go (e.g. water bodies, protected areas, mountains). If supplied and if maskWaterAndMountainsFromLines=TRUE in parameters, the connecting algorithm will treat these areas as high-cost or impassable when routing new roads/pipelines. In the NW Territories case, a raster combining lakes, wetlands and steep slope areas could serve as featuresToAvoid. The model will rasterize vector obstacles if needed to incorporate into pathfinding.
- **DEM (digital elevation model) (optional):** If the useRoadsPackage parameter is TRUE (indicating use of a more advanced road-building algorithm), a DEM raster should be provided to compute slope costs. In the current implementation, this is optional and by default the road connections use simpler distance or straight-line paths unless the user integrates the roads package for more realism.
- **Pre-calculated DisturbanceRate table (optional):** The generator allows an optional input DisturbanceRate (data.table) to be supplied. This would contain pre-set annual disturbance rates per type. If given, the calculatingRate event can

use it instead of computing from diffYears data. In absence of this, the model computes rates internally on BEAD 2010 – BEAD 2015 data.

All the above inputs must be prepared prior to running the simulation. In summary, the minimum inputs to run the model are: a disturbanceDT specifying data sources, and either the data files reachable at the given URLs or already downloaded in the inputPath. Also needed are a study area and raster (defaults can cover this for the provided NT example). The model's modular design means that if certain potential layers are not relevant, the corresponding module can be omitted or the whatNotToCombine parameter can exclude them (for instance, if no special potential processing is needed, one could skip the PotentialResources module and let anthroDisturbance_DataPrep combine everything).

Table 1: Default datasets in use

Dataset	Authority	Years (temporal coverage)	Spatial resolution / geometry	Domain (what it contains)	License	Citation / landing page (APA style)
Anthropogenic disturbance footprint within boreal caribou ranges across Canada (interpreted from 2008–2010 Landsat; updated to 2012 range boundaries)	Environment and Climate Change Canada (ECCC)	2008–2010 imagery baseline (published 2013) (catalogue.ec.gc.ca)	Vector lines + polygons; interpreted from 30 m Landsat at ~1:50,000 viewing scale (methodological description carried through the series) (catalogue.arctic-sdi.org)	National, range-based mapping of anthropogenic linear (e.g., roads, seismic, pipelines, powerlines) and polygonal (e.g., cutblocks/harvest, mines, built-up, wells, agriculture, oil & gas) disturbances	Open Government Licence – Canada (applies to the disturbance-footprint series) (catalogue.arctic-sdi.org)	Environment and Climate Change Canada. (2013). <i>Anthropogenic disturbance footprint within boreal caribou ranges across Canada – As interpreted from 2008–2010 Landsat satellite imagery (Updated to 2012 range boundaries)</i> [Data set]. Government of Canada / ECCC metadata record. (catalogue.ec.gc.ca)
Anthropogenic disturbance footprint within boreal caribou ranges across Canada (2015 update; interpreted from 2015 Landsat)	ECCC	2015 (begin/end dates in metadata: 2015-04-01 to 2015-10-31) (catalogue.arctic-sdi.org)	Vector lines + polygons; nationally consistent mapping using 30 m Landsat and common interpretation rules (series methodology) (catalogue.arctic-sdi.org)	Same disturbance classes as baseline; intended for consistent 5-year updates	Open Government Licence – Canada (catalogue.arctic-sdi.org)	Environment and Climate Change Canada. (2019). <i>2015 – Anthropogenic disturbance footprint within boreal caribou ranges across Canada – As interpreted from 2015 Landsat satellite imagery</i> [Data set]. Government of Canada / Arctic SDI metadata record. (catalogue.arctic-sdi.org)
Anthropogenic disturbance footprint within boreal caribou ranges across Canada (2020 update; interpreted from 2020 Landsat)	ECCC	2020 update (dataset described as the “second 5-year update,” extending the 2015 database to 2020) (catalogue.arctic-sdi.org)	Vector lines + polygons; mapping consistent with series; metadata also documents buffered footprint products (500 m) used for caribou “zone of impact” analyses (catalogue.arctic-sdi.org)	Same disturbance classes; 2020 update adds/retains newly observed features since prior mapping	Open Government Licence – Canada (catalogue.arctic-sdi.org)	Environment and Climate Change Canada. (2024). <i>2020 – Anthropogenic disturbance footprint within boreal caribou ranges across Canada – As interpreted from 2020 Landsat satellite imagery</i> [Data set]. Government of Canada / Arctic SDI metadata record. (catalogue.arctic-sdi.org)
Bird Conservation Regions (BCR) boundaries (used to derive study mask; e.g., BCR6 NT1)	North American Bird Conservation Initiative (NABCI); distributed by Birds Canada	Not time-series (boundary product; versioned by provider)	Vector polygons	Regionalization / study masking (BCR boundaries)	Licence not clearly stated on landing page; provider requests formal citation/acknowledgement (Arctic SDI)	Birds Canada. (n.d.). <i>Bird Conservation Regions</i> [Data set]. NABCI / Birds Canada download page (citation provided by distributor). (Arctic SDI)
National Road Network (NRN) / authoritative roads (used to QA/augment linear infrastructure where applicable)	Statistics Canada / GeoBase (Government of Canada)	Multi-year road network product (versioned releases; module references NRN v13.0 for NT)	Vector linework; national road segments; ISO 14825 referenced in NRN documentation (Birds Canada Oiseaux Canada)	Transportation network (roads and related attributes)	Open Government Licence – Canada (standard GoC open data licensing for NRN releases) (Birds Canada Oiseaux	Statistics Canada. (n.d.). <i>National Road Network (NRN) – GeoBase Series</i> [Data set]. Government of Canada documentation/metadata. (Birds Canada Oiseaux Canada)

Dataset	Authority	Years (temporal coverage)	Spatial resolution / geometry	Domain (what it contains)	License	Citation / landing page (APA style)
					Canada)	
Canadian Wind Turbine Database	Natural Resources Canada (NRCan)	National coverage; updated over time (turbine commissioning varies)	Vector points	Energy infrastructure (wind turbine locations + attributes)	Open Government Licence – Canada (as a GoC open dataset) (Open Government Portal)	Natural Resources Canada. (n.d.). <i>Canadian Wind Turbine Database</i> [Data set]. Government of Canada open data / map record. (Open Government Portal)
NWT Wind Energy Source (layer from GNWT Geomatics services; used as suitability/context)	Government of the Northwest Territories (GNWT), Centre for Geomatics	Not specified (service layer; versioned implicitly)	Map service (raster layer)	Wind potential / energy suitability context	Open Government Licence – Northwest Territories (Government of the Northwest Territories)	Government of the Northwest Territories. (n.d.). <i>NWT Wind Energy Source</i> [Map service layer]. GNWT Centre for Geomatics web services listing. (Geomatics Applications)
Active Mineral Claims (mineral tenure; contextual/QA layer)	GNWT (Mining Recorder's Office / Geomatics services)	Continuously updated (current at download time)	Vector polygons (map service layer)	Mineral tenure (claims) used for context/QA (not a disturbance feature itself)	Open Government Licence – Northwest Territories (Government of the Northwest Territories)	Government of the Northwest Territories. (n.d.). <i>Active Mineral Claims</i> [Map service layer]. GNWT Geomatics (ArcGIS REST service). (Geomatics App Test)
Oil and Gas Rights (permits/licences; contextual/QA layer)	GNWT (Industry, Tourism and Investment / Geomatics services)	Continuously updated (current at download time)	Vector polygons (map service layer)	Petroleum rights areas; context/QA (not a disturbance feature itself)	Open Government Licence – Northwest Territories (Government of the Northwest Territories)	Government of the Northwest Territories. (n.d.). <i>Oil and Gas Rights</i> [Map service layer]. GNWT Geomatics (ArcGIS REST service). (Geomatics Applications)
Transmission Lines (power transmission; infrastructure input/context)	GNWT, Centre for Geomatics	Not specified (service layer; versioned implicitly)	Vector lines (map service layer)	Power transmission infrastructure	Open Government Licence – Northwest Territories (Government of the Northwest Territories)	Government of the Northwest Territories. (n.d.). <i>Transmission Lines</i> [Map service layer]. GNWT Geomatics (ArcGIS REST service). (Geomatics App Test)
Land Use Permit AOIs (e.g., Timberworks; Digaa/MV2015W0018)	Mackenzie Valley Land and Water Board (MVLWB) Public Registry	Permit-specific (varies by file; e.g., MV2015W0018 is a 2015 permit record) (Mackenzie Valley Land and Water Board)	Typically vector polygons (derived from registry documents)	Project/permit AOIs (used as local context layers)	Public registry material; no explicit open data licence identified in the registry entry—treat as “use with attribution; do not redistribute unless permitted” (Mackenzie Valley Land and Water Board)	Mackenzie Valley Land and Water Board. (n.d.). <i>Public Registry (Land Use Permits; e.g., MV2015W0018 and Timberworks Inc. records)</i> [Registry documents]. MVLWB Public Registry. (Mackenzie Valley Land and Water Board)
GNWT boundaries / communities / administrative layers (masking & context)	GNWT, Centre for Geomatics	Varies by layer (service snapshots)	Map services (polygons/points)	Administrative boundaries, community locations, etc.	Open Government Licence – Northwest Territories (Government of the Northwest Territories)	Government of the Northwest Territories. (n.d.). <i>GNWT Geospatial Data and Web Services (boundaries/communities/admin layers)</i> [Data portal and services]. GNWT Centre for Geomatics. (Government of the Northwest Territories)
Elevation (30" ~1 km DEM used for constraints/context; derived via geodata)	Underlying sources: CGIAR-CSI SRTM (void-filled) + USGS GTOPO30 (high latitudes); accessed via R geodata	SRTM acquired 2000; GTOPO30 published 1996; 30" product is aggregated/supplemented	Raster 30 arc-seconds (~1 km)	Elevation baseline for constraint/context (e.g., elevation masks)	Publicly available; use subject to provider terms; attribution expected (R Project Search)	Hijmans, R. J., et al. (n.d.). <i>Elevation data (30 arc-seconds) via geodata; aggregated from CGIAR-CSI SRTM and supplemented with GTOPO30 >60°</i> [Data set / access function]. R package documentation. (R Project Search)
Land cover fractions (water/wetland fractions; derived via geodata from ESA WorldCover)	ESA WorldCover consortium (underlying); accessed via R geodata	WorldCover 2020 (v100) and/or 2021 (v200); fractional product at 30"	Raster 30 arc-seconds (~1 km) fractional cover per class	Fractional land cover used for water/wetland masks and context	CC BY 4.0 (R Project Search)	Zanaga, D., et al. (2021/2022). <i>ESA WorldCover 10 m (2020 v100 / 2021 v200)</i> [Data set]. Zenodo. (Accessed here via geodata fractional 30" products.) (WorldCover)

7. Submodels

Here we detail the submodels corresponding to each major process: data harmonization, potential layer combination, disturbance parameter calculation, disturbance generation, and state updating. Pseudocode or equations are provided where applicable to clarify how the model operates at each step.

7.1 Data Harmonization (Disturbance Data Prep)

Submodel A: Create Disturbance List (`createDisturbanceList`) - *Inputs:* `disturbancesDT.csv` = disturbance metadata table, plus `studyArea`, `rasterToMatch`. *Purpose:* Reads all raw disturbance datasets and produces a structured list of spatial objects. The submodel iterates over each unique combination of `dataName` and `dataClass` in the metadata. For each entry, it performs the following steps:

1. **Data loading:** Use the given URL and `dataType` to download (if not already in cache) and read the spatial data. This is done via `prepInputs()` from the reproducible package, which handles downloading and reading in one step. For a shapefile, this yields a `SpatVector`; for a raster, a `SpatRaster`. If the file is a `.gdb` or other, appropriate functions are used. The function can also unzip files and pick the specified `fileName` inside if provided.

Geometry validity: When vector geometries are encountered during data preparation, the module checks validity and repairs invalid geometries using `terra::makeValid` before continuing. (There is no `skipFixErrors` flag in the current implementation.)

2. **Projection and crop:** The spatial object is reprojected to match `rasterToMatch`'s CRS and then cropped to the `studyArea` polygon. This ensures that only disturbances within the region of interest are kept and that all data layers share the same coordinate system and extent. Rasters are also resampled to the target resolution if needed (for example, if a disturbance dataset came as a coarser raster, it will be upscaled to the resolution of `rasterToMatch`).
3. **Filtering (if needed):** If the metadata specifies a `fieldToSearch` and `classToSearch`, the object's attributes are queried and only features matching the criteria are retained. For instance, if a single shapefile contains multiple disturbance types labeled in an attribute, this filters out only the desired type.
4. **Merging:** If multiple data sources correspond to the same `dataClass`, they are merged. For vectors, this means concatenating the features; for rasters, possibly

mosaic or stacking (though typically each class is either all vector or all raster). The goal is to end up with one combined layer per class. For example, if disturbanceDT had separate entries for "Main Roads" and "Secondary Roads" both mapped to dataClass = roads, after loading both, the submodel would combine them into one roads vector layer.

5. **Output structuring:** The processed layer is placed into a nested list structure: `disturbances[[dataName]][[dataClass]] <- spatialLayer`. Here dataName is the sector grouping (like "Forestry") and dataClass is the specific type (like "cutblocks"). Thus, disturbances is a list of lists. If multiple classes belong to the same sector, they end up in the same top-level list.

This submodel ensures all input data are normalized to a common spatial domain and format. After this, the model has a raw disturbances list where some classes may still have multiple entries (if not merged yet) or might include "potential" classes that mirror current ones (like an entry for current mines and one for potential mines).

Submodel B: Harmonize List (harmonizeList) - *Inputs:* disturbances list from above, and whatNotToCombine parameter. *Purpose:* Finalize the disturbanceList by unifying duplicate classes and segregating special cases. The algorithm goes through each sector in disturbances and for each dataClass inside it:

- If the dataClass name contains any substring listed in whatNotToCombine (by default this list contains "potential"), then that class is excluded from combination. This means if, for example, there are multiple "potentialSomething" layers, they will remain separate or be handled in another module. In the NW Territories case, whatNotToCombine = "potential" ensures that classes like "potentialMining" and "potentialOilGas" are not merged with other classes (since they need idiosyncratic processing).
- For all other classes, if a sector has more than one list element for the same class (which could happen if the data sources were split by region or year), they are merged into one. In many cases, createDisturbanceList already merged by class, so harmonizeList primarily just copies the structure. The outcome is a new list disturbanceList where each (sector, class) combination is unique and holds a single spatial layer. Essentially it collapses disturbances into a cleaner form.

The harmonizeList returns `sim$disturbanceList`. After Submodel A and B, the data are ready for use by the simulation. For example, `disturbanceList$Forestry$cutblocks` might

be a polygon layer of all existing cutblocks in the study area; `disturbanceList$Energy$windTurbines` might be point locations of wind farms; `disturbanceList$Energy$potentialWind` might be a raster of wind power potential (this one would be left separate for potential processing if needed).

7.2 Potential Resource Layer Combination (Idiosyncratic Data Prep)

This submodel (implemented in the `potentialResourcesNT_DataPrep` module for the NT case) handles special rules to combine multiple data sources into single “potential” layers for future development. The need for this arises when the way new disturbances should be generated cannot rely on a single raw input layer. Two key examples are Oil & Gas and Mining potential, where multiple datasets (e.g. geological survey outputs, current exploration activities) must be fused.

Submodel C: Create Combined Potential Layers - The module runs a series of events at initialization:

- **createPotentialMining:** This uses the `whatToCombine` table (see Input Data) to identify which `disturbanceList` entries to combine for mining. In the NT example, `whatToCombine` indicates that for `dataName = "mining"` and `dataClass = "potentialMining"`, two inputs are relevant: one with `toDifferentiate = "CLAIM_STAT"` and one with `toDifferentiate = "PERMIT_STA"`. Likely, the disturbance list at this point has two raster layers (or Spatial tables) for mining potential: one showing areas of active claims and one of permits. The submodel reads those from `sim$disturbanceList` and creates a new raster that assigns higher values to cells with active claims, slightly lower to cells with only permits, and baseline values to other areas (perhaps based on an underlying geological potential if available). In practice: it might take the “CLAIM_STAT” layer (which could be a polygon of mineral claims) and rasterize it, giving those cells a high score (say 1.0), take the “PERMIT_STA” (exploration permit areas) and give them a moderate score (say 0.5), and everywhere else 0. Then maybe add a base geological favorability map if provided. The output is a raster with values from high (very likely to develop - active claims) to low (no interest). This becomes `sim$potentialMining`.
- **CreatePotentialOilGas, createPotentialSeismicLines, createPotentialCutblocks:** all follow a similar path to **createPotentialMining**.

Each of these `create*` functions returns a new spatial layer which the module stores in `sim` (e.g., `sim$potentialMining`).

- **replaceInDisturbanceList:** After creating the new layers, this final step updates the main disturbanceList. For each combined potential (mining, oilGas, seismic, cutblocks), it finds the corresponding entries in disturbanceList and replaces them. For example, originally disturbanceList\$Mining might have had two entries: claims and permits (or a placeholder potentialMining that was not yet combined). Now it will have one entry potentialMining (the combined raster) and presumably no separate permits/claims entries (those could be removed).

After this submodel, the disturbanceList is fully updated with refined potential layers. This completes the initialization phase. The generator module now has everything it needs: a disturbanceList with current disturbances and potential surfaces, and the disturbance parameters table.

7.3 Disturbance Parameter Calculations

At the beginning of the simulation, the module checks whether each simulated disturbance class has the two key parameters needed to generate change: a representative disturbanceSize used when creating new features, and an annual disturbanceRate expressed as a percentage of the study area per year. If these values are already specified in the disturbanceParameters table, they are left unchanged. Otherwise, they are derived from available spatial data or from user-provided scenario inputs.

Submodel D: Calculate Disturbance Size (calculateSize) — *Purpose:* Populate disturbanceParameters\$disturbanceSize for Generating rows when it is missing. For each generating class needing a size, the model retrieves the corresponding existing layer from disturbanceList[[dataName]][[disturbanceOrigin]]. Polygon feature sizes are computed as areas (m²) and line feature sizes are computed as lengths (m). The model then stores a truncated normal sampling expression as a string, using `msm::rtnorm(1, mean, sd, lower = 0)` (with defensively-chosen fallbacks when the empirical variance is degenerate). If the required existing layer is missing/empty, the model cannot infer a size and that disturbance class will not be simulated unless the user supplies a size manually.

Submodel E: Calculate Disturbance Rate (calculateRate) — *Purpose:* Populate disturbanceParameters\$disturbanceRate for Generating and Enlarging rows when it is missing. Rates are expressed as annual percentages of the study area. If the user supplies a DisturbanceRate table, those values are used directly. If the user supplies totalDisturbanceRate, the model uses empirically-derived proportions to allocate the

total rate across disturbance classes. If neither is supplied, the model derives per-class annual change rates from external reference footprint layers (via the ECCC comparison workflow) for the configured diffYears. If a class shows negative or absent change in the reference comparison, its rate is set to 0 (disturbance reduction is not simulated).

7.4 Disturbance Generation Dynamics

This set of submodels governs how disturbances grow or appear each simulated interval. At each simulation step, the generator reads the disturbanceParameters table and the current disturbanceList to identify which disturbance classes are active for the current year (based on disturbanceInterval and disturbFirstYear). It then creates year-specific updates in three stages: (F) enlarging the footprint of existing features, (G) generating new features in potential areas, and (H) connecting new origins to existing networks. All stages are constrained to the studyArea and use rasterToMatch to keep a consistent spatial resolution; a cumulative disturbance mask (currentDisturbanceLayer) prevents re-selecting already disturbed cells.

Submodel F: Enlarging existing disturbances.

For each disturbance class with disturbanceType = “Enlarging” that is active in the current year, the model computes a target footprint increase and expands the existing layer until that target is reached:

1. Determine the target increase. The annual disturbanceRate is translated to the current time step by multiplying by disturbanceInterval and the study-area size, yielding a target added footprint (area or an area-equivalent for linear features). If disturbanceRateRelatesToBufferedArea = TRUE, both target and achieved footprint are evaluated on buffered geometries (e.g., 500 m buffers), which aligns with footprint-based observational products.
2. Iteratively expand the existing layer. The model applies small buffer increments (growthStepEnlargingLines for line geometries; growthStepEnlargingPolys for polygon geometries) and unions the result with the previous footprint. After each increment, the newly added footprint is re-evaluated against the target.
3. Stop criteria and constraints. Expansion stops when the target increase is met or when the available undisturbed area is exhausted. Minor overshoot can occur because buffering is discrete; the realized increase is recorded for diagnostics.

The output of Submodel F is an updated layer for each enlarging class, returned as part of the Enlarged component of the generator output.

Submodel G: Generating new disturbances (site selection and feature creation).

Generating disturbances (default vector workflow): For each Generating class scheduled at the current time step, the model converts the annual disturbanceRate into a target *new disturbed area* for this step:

$\text{target_area} = (\text{disturbanceRate}/100) * \text{area}(\text{studyArea}) * \text{disturbanceInterval}$. New disturbances are then placed inside the class's potential layer (a polygon suitability surface with a Potential attribute). Site selection proceeds by preferentially using higher-potential polygons; optionally, for disturbances listed in siteSelectionAsDistributing, selection is probabilistic across potential polygons rather than strictly "highest-first". For most disturbance types, already-disturbed areas are removed from the candidate potential polygons (using the combined current disturbance layers) so new features are not placed on top of existing disturbance footprints.

Each new feature is created by sampling a target size from the class's disturbanceSize expression and sampling a location within the eligible potential polygons. A polygon footprint of the requested area is generated around the sampled location. This repeats until the accumulated new disturbed area meets the target for the time step (within iteration limits and feasibility constraints).

Raster-mode option: The generator also supports a raster-based workflow (generatedDisturbanceAsRaster = TRUE) that works by selecting raster cells to satisfy target rates. In this project configuration the default workflow is vector-based (generatedDisturbanceAsRaster = FALSE), because it better supports sub-pixel feature sizes and vector connectivity.

Seismic lines: In the default parameterization, seismic lines are treated as a Generating disturbance class with its own siting and geometry rules. Seismic line generation always uses buffered-area accounting for rate matching (i.e., it is evaluated using a 500 m buffer regardless of the global disturbanceRateRelatesToBufferedArea flag). New seismic lines are generated within the seismic potential layer (in the NT workflow this potential is derived from oil/gas potential). Two supported approaches are: a clustering/duplication approach that characterizes existing seismic line "grids" and simulates new line networks with similar structure, and a direct grid-construction

approach that draws grid-like line patterns inside sampled potential areas. In both cases, simulated lines are buffered to an approximate line width (e.g., ~3 m) to represent their footprint.

Unlike other disturbance types, seismic line generation may allow overlap with existing seismic lines (i.e., candidate potential areas are not strictly erased by current seismic lines), because the process represents repeated line cutting within high-activity zones.

Connecting disturbances: For each Connecting class, the model connects newly generated origin features (e.g., new mines or oil/gas sites) to an existing destination network layer (e.g., roads, pipelines, power lines). By default, the connection geometry is built using nearest-neighbour connections to the destination network (straight-line connectors). If `maskWaterAndMountainsFromLines = TRUE`, the model instead performs grid-based routing on a cost raster derived from `rasterToMatch`, treating `featuresToAvoid` as impassable cells, and generates connecting lines that avoid those obstacles. Optionally, `connectingBlockSize` can be used to process large numbers of origins in blocks for performance; smaller blocks generally reduce “many origins connected at once” artefacts at the expense of longer run times. A separate optional pathway exists for the roads package (`useRoadsPackage = TRUE`), which can use a DEM-derived surface as input to road routing.

7.5 State update and outputs

The generator returns a list with (at minimum) two components:

- (1) `individualLayers` — the newly updated disturbance layers produced during the current step, organized as a nested list by sector and class (primarily `SpatVectors`).
- (2) `currentDisturbanceLayer` — a nested list of disturbance layers used for masking/exclusion in subsequent steps (with buffering/conversion to polygons applied where needed for consistent intersection/erase operations).

In addition, the generator may return a seismic-lines baseline object for the first year to support consistent merging of seismic lines across time.

State update (`replaceListFast`): After each generation step, the model integrates the returned layers into `sim$disturbanceList`. Disturbance classes marked `Enlarging` (with `disturbanceEnd == ""`) replace the previous layer for that class. For other classes, newly generated/connected features are appended to existing features (preserving earlier features) and are tagged with a `createdInSimulationTime` attribute. Potential layers are

preserved and re-attached to the sector list after updating the current-disturbance layers.

Saved outputs: When enabled, the model saves per-sector, per-class disturbance layers to disk. In the default vector workflow, layers are written as vector files (e.g., shapefiles). Raster outputs are written only when the object being saved is a raster. The workflow may also write a buffered (e.g., 500 m) disturbance raster summary for the initial conditions when requested.

Finally, the state-update event schedules the next disturbance-generation cycle for the subsequent time step, repeating the enlarge-generate-connect-update sequence until the end of the simulation horizon.

References: Grimm et al. (2006; 2010; 2020) ODD Protocol; Railsback & Grimm (2019) pattern-oriented modeling concepts; Micheletti (2025) SpaDES anthropogenic disturbance modules (Appendices A-C).